601.433/633 Introduction to Algorithms **Fall 2026**
Homework #6 **Due:** October 29, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Computer Viruses (50 points)

You're helping some security analysts monitor a computer network, tracking the spread of an online virus. There are n computers in the system, labeled $C_1, C_2, \ldots, C_n$. You are given a collection of *trace data* indicating the times at which pairs of computers communicated. This data is a sequence of ordered triples (C_i, C_j, t_k), where such a triple indicates that C_i and C_j exchanged bits at time t_k. There are m triples total.

We'll assume that the triples are presented to you in sorted order of time, and for simplicity we'll assume that each pair of computers communicates at most once during the interval that you're observing.

The security analysts you're working with would like to able to answer questions of the following form: if the virus was inserted into computer C_a at time 0, what is the first time at which it could have have infected computer C_b? If it cannot be infected at all, then we say that this time is ∞.

The mechanics of infection are simple: if an infected computer C_i communicates with an uninfected computer C_j at time T_k (i.e., if one of the triples (C_i, C_j, t_k) or (C_j, C_i, t_k) appears in the trace data), then computer C_j becomes infected as well, starting at time t_k. Infection can thus spread from one machine to another across a *sequence* of communications, provided that no step in this sequence involves a move backward in time. Thus, for example, if C_i is infected by time t_k, and the trace data contains triples (C_i, C_j, t_k) and (C_j, C_q, t_r) where $t_k \leq t_r$, then C_q will become infected via C_j. Note that it is OK for t_k to be equal to t_r; this would mean that C_j had open connections to both C_i and C_q at the same time, and so a virus could move from C_i to C_q through C_j.

For example, suppose that $n = 4$ and the trace data consists of the triples

$$(C_1, C_2, 4), (C_2, C_4, 8), (C_3, C_4, 8), (C_1, C_4, 12)$$

and the virus was inserted into computer C_1 at time 0. Then C_3 would first be infected at time 8 by a series of three steps: first C_2 becomes infected at time 4, then C_4 from C_2 at time 8, and then C_3 from C_4 at time 8. So we should return 8 if the query is C_3.

On the other hand, if the trace data were

$$(C_2, C_3, 8), (C_1, C_4, 12), (C_1, C_2, 14)$$

and again the virus was inserted into C_1 at time 0, then C_3 would *not* become infected during the period of observation. So we should return ∞ if the query is C_3.

So your goal is to design an algorithm (as fast as possible) that answers questions of this type. The input is a collection of trace data (consisting of m triples), a virus injection point (a computer C_a), and another computer C_b. Your algorithm must return the first time at which C_b could be infected, assuming the virus was injected into C_a at time 0.

As always, prove correctness and running time.

Algorithm Description:
TODO

Correctness Proof:
TODO

Runtime Proof:
TODO

2 Consistent Lifetimes (50 points)

Suppose that you are an historian, and you are trying to figure out possible dates for when various historical figures may have lived. In particular, there are n people $P_1, P_2, \ldots, P_n$ who you are studying. Based on your research, you have discovered a collection of m facts about when these people lived relative to each other. Each of these facts has one of the following two forms:

- For some i and j, person P_i died before person P_j was born; or
- For some i and j, the lifetimes of P_i and P_j overlapped.

Unfortunately, since the historical record is never fully trustworthy, it's possible that some of these facts are incorrect. Design an $O(n + m)$ time algorithm to at least check whether they are *internally consistent*, i.e., whether there is a birth and a death date for each person so that all of the facts are true. As always, prove running time and correctness (i.e., prove that if there is a possible set of dates then your algorithm will return yes, and if no such dates are possible then your algorithm will return no).

Algorithm Description:
TODO

Correctness Proof:
TODO

Runtime Proof:
TODO